18. CLI 구조 분석

개요

Omni CLI는 Omni 네트워크 생태계의 운영자와 개발자를 위한 명령줄 도구입니다. 이 문서는 CLI의 구조와 주요 기능을 상세히 분석합니다.

CLI 아키텍처

진입점 구조

```
// cli/cmd/omni/main.go

func main() {

cmd := clicmd.New()

libcmd.WrapRunE(cmd, func(ctx context.Context, err error) {

if cliErr := new(clicmd.CliError); errors.As(err, &cliErr) {

// 사용자 친화적인 CLI 오류 메시지와 제안사항 표시

log.Error(ctx, "❌ "+cliErr.Error(), nil)

} else {

log.Error(ctx, "❌ Error", err)

}

})
```

```go
fig := figure.NewFigure("omni", "", true)

cmd.SetHelpTemplate(fig.String() + "\n" + cmd.HelpTemplate())


ctx := log.WithCLILogger(context.Background())

err := cmd.ExecuteContext(ctx)

if err != nil {

os.Exit(1)

}

}
```

루트 명령어 구조

```go
// cli/cmd/cmd.go

func New() *cobra.Command {

return libcmd.NewRootCmd(

"omni",

"CLI providing tools for interacting with omni",

newOperatorCmds(), // 운영자 명령어

newDeveloperCmds(), // 개발자 명령어

newDevnetCmds(), // 로컬 개발 네트워크 명령어

buildinfo.NewVersionCmd(), // 버전 정보

)

}
```

주요 명령어 그룹

1. 운영자 명령어 (operator)

검증자 및 네트워크 운영을 위한 명령어들:

```
func newOperatorCmds() *cobra.Command {
cmd := &cobra.Command{
Use: "operator",
Short: "Operator commands",
Args: cobra.NoArgs,
}

cmd.AddCommand(
newInitCmd(), // 노드 초기화
newCreateValCmd(), // 검증자 생성
newCreateOperatorKeyCmd(), // 운영자 키 생성
newCreateConsensusKeyCmd(), // 합의 키 생성
newUnjailCmd(), // 검증자 언제일
newDelegateCmd(), // 위임
newEditValCmd(), // 검증자 정보 수정
)

return cmd
```

```
}
```

1.1 노드 초기화 (init-nodes)

```
type InitConfig struct {
Network netconf.ID // 네트워크 ID (mainnet, testnet, devnet)
Home  string  // 홈 디렉토리 경로
Moniker  string  // 노드 이름
Clean  bool  // 기존 데이터 삭제 여부
Archive  bool  // 아카이브 모드 활성화
Debug  bool  // 디버그 로그 레벨
NodeSnapshot  bool  // 스냅샷 다운로드 여부
HaloTag  string  // Halo Docker 이미지 태그
}
```

주요 기능:

- Geth 실행 노드 초기화
- Halo 합의 노드 초기화
- Docker Compose 파일 생성
- 제네시스 파일 다운로드
- 스냅샷 복원 (선택적)

1.2 검증자 관리

검증자 생성 (create-validator):

```go
type CreateValConfig struct {
EOAConfig
ConsensusPubKeyHex string // 합의 공개키 (hex)
SelfDelegation uint64 // 자기 위임 금액 (최소 100 OMNI)
}
```

위임 (delegate):

```go
type DelegateConfig struct {
EOAConfig
Amount uint64 // 위임 금액 (최소 1 OMNI)
Self bool // 자기 위임 여부
ValidatorAddress string // 대상 검증자 주소
}
```

검증자 정보 수정 (edit-validator):

```go
type EditValConfig struct {
EOAConfig
Moniker string // 검증자 이름
Identity string // 신원 정보
Website string // 웹사이트
SecurityContact string // 보안 연락처
Details string // 상세 설명
```

```
CommissionRatePercentage int32 // 수수료율 (%)
MinSelfDelegationEther int64 // 최소 자기 위임 (OMNI)
}
```

1.3 키 관리

운영자 키 생성:

```
func newCreateOperatorKeyCmd() *cobra.Command {
// EOA 개인키 생성 및 저장
privKey, err := crypto.GenerateKey()
addr := crypto.PubkeyToAddress(privKey.PublicKey).Hex()
privKeyfile := strings.Replace(cfg.PrivateKeyFile, "{ADDRESS}", addr, 1)
if err := crypto.SaveECDSA(privKeyfile, privKey); err != nil {
return errors.Wrap(err, "save key")
}
}
```

합의 키 생성:

```
func newCreateConsensusKeyCmd() *cobra.Command {
// CometBFT 합의 개인키 및 상태 파일 생성
filePV := privval.NewFilePV(k1.GenPrivKey(), keyFile, stateFile)
filePV.Save()
// 투표자 상태 파일 생성
```

```go
if err := voter.GenEmptyStateFile(voterFile); err != nil {
return err
}
}
```

2. 개발자 명령어 (developer)

XApp 개발을 위한 도구들:

```go
func newDeveloperCmds() *cobra.Command {
cmd := &cobra.Command{
Use: "developer",
Short: "XApp development commands",
Args: cobra.NoArgs,
}

cmd.AddCommand(
newForgeProjectCmd(), // Forge 프로젝트 스캐폴딩
)

return cmd
}
```

2.1 프로젝트 스캐폴딩 (new)

```go
type developerForgeProjectConfig struct {
templateName  string  // 템플릿 이름 (기본: hello-world-template)
}


func  newForgeProjectTemplate(ctx context.Context, cfg developerForgeProjectConf
// 1. GitHub에서 템플릿 저장소 클론
cmd := exec.CommandContext(ctx, "git", "clone",
"https://github.com/omni-network/"+cfg.templateName+".git", destinationPath)
// 2. 특정 커밋 체크아웃 (기본 템플릿의 경우)
if  cfg.templateName == defaultTemplate {
cmd = exec.CommandContext(ctx, "git", "checkout", defaultTemplateCommit)
}
// 3. 서브모듈 초기화
cmd = exec.CommandContext(ctx, "git", "submodule", "update", "--init", "--recurs
// 4. .git 디렉토리 삭제 및 새 저장소 초기화
os.RemoveAll(filepath.Join(destinationPath, ".git"))
cmd = exec.CommandContext(ctx, "git", "init")
}
```

3. 로컬 개발 네트워크 명령어 (devnet)

로컬 테스트 환경 관리:

```go
func newDevnetCmds() *cobra.Command {
cmd := &cobra.Command{
Use: "devnet",
Short: "Local devnet commands",
Args: cobra.NoArgs,
}

cmd.AddCommand(
newDevnetFundCmd(), // 계정 자금 지원
newDevnetStartCmd(), // 개발 네트워크 시작
newDevnetInfoCmd(), // 네트워크 정보 표시
newDevnetCleanCmd(), // 네트워크 정리
)

return cmd
}
```

3.1 개발 네트워크 시작 (start)

```go
func deployDevnet(ctx context.Context) error {
def, err := devnetDefinition(ctx)
if err != nil {
return err
```

```
}

_, err = app.Deploy(ctx, def, app.DefaultDeployConfig())
return err
}
```

네트워크 구성:

- 2개의 Anvil 노드 (EVM 체인)
- 1개의 Halo 노드 (합의 체인)
- Docker Compose 기반 배포

3.2 계정 자금 지원 (fund)

```
type devnetFundConfig struct {
Address string // 자금을 받을 주소
RPCURL string // RPC 엔드포인트
}

func devnetFund(ctx context.Context, cfg devnetFundConfig) error {
funder, backend, err := devnetBackend(ctx, cfg.RPCURL)
addr := common.HexToAddress(cfg.Address)
tx, _, err := backend.Send(ctx, funder, txmgr.TxCandidate{
To: &addr,
GasLimit: 100_000,
```

```
Value: bi.Ether(1), // 1 ETH 전송
})
}
```

3.3 네트워크 정보 표시 (info)

```
func printDevnetInfo(ctx context.Context) error {
network, endpoints, err := loadDevnetNetwork(ctx)
type info struct {
ChainID uint64 `json:"chain_id"`
ChainName string `json:"chain_name"`
PortalAddress common.Address `json:"portal_address"`
RPCURL string `json:"rpc_url"`
}
// JSON 형태로 체인 정보 출력
jsonOutput, err := json.MarshalIndent(infos, "", " ")
fmt.Println(string(jsonOutput))
}
```

설정 및 플래그 관리

EOA 설정

```
type EOAConfig struct {
```

```
Network netconf.ID // 네트워크 ID
ExecutionRPC  string  // 실행 RPC 엔드포인트
ConsensusRPC  string  // 합의 RPC 엔드포인트
PrivateKeyFile  string  // 개인키 파일 경로
}

func (v EOAConfig) privateKey() (*ecdsa.PrivateKey, error) {
opPrivKey, err := crypto.LoadECDSA(v.PrivateKeyFile)
if  err != nil {
return  nil, errors.Wrap(err, "load private key")
}
return  opPrivKey, nil
}
```

플래그 바인딩

```
func  bindEOAConfig(cmd *cobra.Command, cfg *EOAConfig) {
bindPrivateKeyFile(cmd, &cfg.PrivateKeyFile)
netconf.BindFlag(cmd.Flags(), &cfg.Network)
cmd.Flags().StringVar(&cfg.ExecutionRPC, "execution-rpc", "",
"Optional Omni EVM execution RPC API endpoint")
cmd.Flags().StringVar(&cfg.ConsensusRPC, "consensus-rpc", "",
"Optional Omni consensus RPC API endpoint")
```

```
_ = cmd.MarkFlagRequired(flagNetwork)

}
```

오류 처리

사용자 친화적 오류 메시지

```
type CliError struct {

Msg  string  // 오류 메시지

Suggest  string  // 제안사항

Attrs []any // 추가 속성

}


func (e *CliError) Error() string {

var  sb strings.Builder

_, _ = sb.WriteString(e.Msg)

if  e.Suggest != "" {

_, _ = sb.WriteString("\n")

_, _ = sb.WriteString("🤔 " + e.Suggest)

}

return  sb.String()

}
```

사용 예시:

```
if !ok {
return &CliError{
Msg: "Operator address not a validator: " + opAddr.Hex(),
Suggest: "Ensure operator address is a validator",
}
}
```

클라이언트 설정

다중 클라이언트 설정

```
func  setupClients(
ctx context.Context,
conf EOAConfig,
operatorPriv *ecdsa.PrivateKey,
) (ethclient.Client, cchain.Provider, *ethbackend.Backend, error) {
// 1. 합의 클라이언트 설정
cl, err := http.New(conf.ConsensusRPC, "/websocket")
cprov := provider.NewABCI(cl, conf.Network)
// 2. 실행 클라이언트 설정
eth, err := ethclient.DialContext(ctx, chainMeta.Name, conf.ExecutionRPC)
// 3. 백엔드 설정 (트랜잭션 관리)
```

```
backend, err := ethbackend.NewBackend(
chainMeta.Name,
chainID,
chainMeta.BlockPeriod,
eth,
operatorPriv,
)
return  eth, cprov, backend, err
}
```

Docker 통합

Compose 파일 생성

```
func  writeComposeFile(ctx context.Context, cfg InitConfig, upgrade  string) err
tmpl, err := template.New("compose").Parse(string(composeTpl))
err = tmpl.Execute(&buf, struct {
HaloTag  string
GethTag  string
GethVerbosity  int
GethArchive  bool
GenesisBinary  string
}{
```

```
HaloTag: cfg.HaloTag,
GethTag: geth.ServerVersion,
GethVerbosity: verbosity,
GethArchive: cfg.Archive,
GenesisBinary: upgrade,
})
}
```

Geth 초기화

```
func  gethInit(ctx context.Context, cfg InitConfig, dir  string) error {
// 1. 제네시스 파일 작성
genesisJSON := cfg.Network.Static().ExecutionGenesisJSON
os.WriteFile(genesisFile, genesisJSON, 0o644)
// 2. 설정 파일 작성
gethCfg := geth.Config{
Moniker: cfg.Moniker,
ChainID: cfg.Network.Static().OmniExecutionChainID,
IsArchive: cfg.Archive,
BootNodes: bootnodes,
}
geth.WriteConfigTOML(gethCfg, configFile)
// 3. JWT 시크릿 생성
```

```
secret := hex.EncodeToString(k1.GenPrivKey().Bytes())

os.WriteFile(secretFile, []byte(secret), 0o666)

// 4. Docker를 통한 Geth 초기화

dockerArgs := []string{"run",

"-v", dir + ":/geth",

image, "--",

"init",

"--datadir=/geth",

"--state.scheme=" + stateScheme,

"/geth/genesis.json",

}

cmd := exec.CommandContext(ctx, "docker", dockerArgs...)

}
```

스냅샷 관리

병렬 스냅샷 다운로드

```
func  maybeDownloadSnapshots(ctx context.Context, cfg InitConfig) error {

if !cfg.NodeSnapshot {

return  nil

}
```

```
g, ctx := errgroup.WithContext(ctx)

// 병렬 다운로드 시작
g.Go(func() error {
return  downloadSnapshot(ctx, cfg.Network, cfg.Home, gethClientName)
})

g.Go(func() error {
return  downloadSnapshot(ctx, cfg.Network, cfg.Home, haloClientName)
})

// 모든 다운로드 완료 대기
return  g.Wait()
}

func  downloadSnapshot(ctx context.Context, network netconf.ID, outputDir  strin
gcpCloudStorageURL := fmt.Sprintf(
"https://storage.googleapis.com/%s-node-snapshots-archive/%s_data.tar.lz4",
network, clientName)
return  downloadUntarLz4(ctx, gcpCloudStorageURL, filepath.Join(outputDir, clien
}
```

검증 및 보안

입력 검증

```
func (c CreateValConfig) validate() error {
if err := c.EOAConfig.validate(); err != nil {
return err
}

if _, err := c.consensusPublicKey(); err != nil {
return errors.Wrap(err, "verify --consensus-pubkey-hex flag")
}

if c.SelfDelegation < minSelfDelegation {
return errors.New("insufficient --self-delegation",
"minimum", minSelfDelegation, "self_delegation", c.SelfDelegation)
}

return nil
}
```

허용 목록 확인

```
func CreateValidator(ctx context.Context, cfg CreateValConfig) error {
```

```
// 허용 목록이 활성화된 경우 검증자 허용 여부 확인
if enabled, err := contract.IsAllowlistEnabled(bindOpts); err != nil {
return err
} else if enabled {
if ok, err := contract.IsAllowedValidator(bindOpts, opAddr); err != nil {
return err
} else if !ok {
return &CliError{
Msg: "Operator address not allowed to create validator: " + opAddr.Hex(),
Suggest: "Contact Omni team to be included in validator allow list",
}
}
}
}
```

사용법 예시

노드 초기화

```
omni operator init-nodes --network=testnet --moniker="my-node" --clean
```

검증자 생성

```
omni operator create-validator \
--network=testnet \
--private-key-file=./operator.key \
--consensus-pubkey-hex=03a1b2c3... \
--self-delegation=100
```

개발 네트워크 시작

```
omni devnet start
omni devnet info
omni devnet fund --rpc-url=http://localhost:8545 --address=0x123...
```

프로젝트 스캐폴딩

```
omni developer new --template=hello-world-template
```

결론

Omni CLI는 네트워크 운영자와 개발자를 위한 포괄적인 도구 세트를 제공합니다. 주요 특징은:

1. **모듈화된 구조**: 운영자, 개발자, 개발 네트워크별로 명령어 그룹화
2. **사용자 친화적**: 명확한 오류 메시지와 제안사항 제공
3. **Docker 통합**: 컨테이너 기반 배포 및 초기화 지원
4. **보안 중심**: 키 관리 및 검증 기능 내장

5. **개발 지원**: 프로젝트 템플릿 및 로컬 테스트 환경 제공

이러한 기능들을 통해 Omni 네트워크의 운영과 개발을 효율적으로 지원합니다.